SmartNest MQTT & Cloud Integration Guide

Prepared for the cloud/backend developer.

Source of truth: current local repository firmware and documentation inspected on 2026-06-24. Files inspected include ‘SmartNest/SmartNest.ino‘, ‘master/master.ino‘, ‘slave_digital_board/slave_digital_board.ino‘, ‘slave_pzem/slave_pzem.ino‘, and the existing files under ‘docs/‘.

1. Project Overview

SmartNest uses several ESP32 boards with clear ownership boundaries.

```
| Component | Responsibility |
|---|---|
| Internet ESP32 / SmartNest ESP32 | Wi-Fi, MQTT connection, local dashboard, cloud communication, local relay 1-6 control, DHT11 sensor, local ACS current sensing, UART communication with Master ESP32 |
| Master ESP32 | Energy calculation, SD card logging, daily reset logic, RTC/NTP time handling, ESP-NOW communication with slave boards, telemetry forwarding to the Internet ESP32 |
| Digital Board slave | Relay 7, relay 7 manual switch, ACS current sensing for relay 7, relay lock, overcurrent protection |
| PZEM slave | PZEM-004T voltage/current/power/energy telemetry, PZEM reboot, PZEM energy reset command |
```

Live data path:

```
Sensors / slaves -> Master ESP32 -> UART -> Internet ESP32 -> MQTT broker -> Cloud
```

Offline history path:

```
Master SD card -> UART history response -> Internet ESP32 -> MQTT history batch -> Cloud ACK -> Master sync_state.txt update and row purge
```

The Master owns energy history. It writes ‘energy_log.csv‘ to SD first. The Internet ESP32 only requests unsent rows, packages them into MQTT history batches, and forwards cloud acknowledgements back to the Master.

2. MQTT Configuration

Actual MQTT defaults in ‘SmartNest/SmartNest.ino‘:

```
| Setting | Current firmware value |
|---|---|
| MQTT enabled default | ‘true‘ |
| Broker | ‘broker.hivemq.com‘ |
| Port | ‘1883‘ |
| Client ID | ‘SmartNest_001‘ |
| Username | empty string |
| Password | empty string |
```

```
| Keepalive | ‘60‘ seconds |
| Base topic | ‘smartnest/SmartNest_001‘ |
| History batch limit requested by Internet ESP32 | ‘6‘ records |
| History retry / command timeout | ‘15000‘ ms |
| MQTT max history payload size | ‘2800‘ bytes |
```

Identity rules:

- Client ID is firmware-fixed/read-only.
- Base topic is firmware-fixed/read-only.
- ‘loadMqttConfig()‘ always restores client ID from ‘MQTT_CLIENT_ID‘ and base topic from ‘MQTT_BASE_TOPIC‘.
- ‘saveMqttConfig()‘ does not save client ID or base topic to preferences.
- ‘MQTT RESET‘ restores the fixed default client ID and base topic.

Dashboard editable settings:

- ‘enabled‘
- ‘broker‘
- ‘port‘
- ‘user‘
- ‘pass‘
- ‘keepalive‘

Dashboard readonly settings:

- ‘client‘
- ‘topic‘ / ‘baseTopic‘
- publish topic list
- subscribe topic list

‘GET /api/mqtt‘ returns:


```
{
  "enabled": true,
  "broker": "broker.hivemq.com",
  "port": 1883,
  "client": "SmartNest_001",
  "user": "",
  "pass": "",
  "topic": "smartnest/SmartNest_001",
  "baseTopic": "smartnest/SmartNest_001",
  "client_readonly": true,
  "topic_readonly": true,
  "publish_topics": [
    "smartnest/SmartNest_001/live/sensors",
    "smartnest/SmartNest_001/live/relays",
    "smartnest/SmartNest_001/live/status",
    "smartnest/SmartNest_001/history/batch",
    "smartnest/SmartNest_001/cmd/ack"
  ],
  "subscribe_topics": [
    "smartnest/SmartNest_001/cmd/request",
    "smartnest/SmartNest_001/history/ack"
  ],
  "keepalive": 60
```

```
}
```

‘POST /api/mqtt‘ accepts and saves only:

- ‘enabled‘
- ‘broker‘
- ‘port‘
- ‘user‘
- ‘pass‘
- ‘keepalive‘

Rejected through ‘POST /api/mqtt‘:

```
| Submitted field | If changed | Response |
|---|---|---|
| ‘client‘ | differs from fixed client ID | ‘ERR: MQTT client ID is read-only‘ |
| ‘topic‘ | differs from fixed base topic | ‘ERR: MQTT topic is read-only‘ |
| ‘baseTopic‘ | differs from fixed base topic | ‘ERR: MQTT topic is read-only‘ |
```

Serial/dashboard command restrictions:

```
| Command | Current behavior |
|---|---|
| ‘MQTT SET CLIENT <clientId>‘ | Rejected: ‘ERR: MQTT client ID is fixed/read-only‘ |
| ‘MQTT SET TOPIC <baseTopic>‘ | Rejected: ‘ERR: MQTT topic is fixed/read-only‘ |
| ‘MQTT SET BROKER <host>‘ | Implemented |
| ‘MQTT SET PORT <port>‘ | Implemented |
| ‘MQTT SET USER <username>‘ | Implemented |
| ‘MQTT SET PASS <password>‘ | Implemented |
| ‘MQTT SET KEEPALIVE <seconds>‘ | Implemented |
| ‘MQTT ENABLE ON/OFF‘ | Implemented |
| ‘MQTT SHOW‘ | Implemented |
| ‘MQTT RESET‘ | Implemented |
```

Not implemented:

- MQTT TLS.
- MQTT retained publishes.
- Topic editing from dashboard/API/serial command.
- ‘/api/logs/list‘, ‘/api/logs/clear‘, and ‘/api/logs/energy.csv‘ routes in the current ‘SmartNest.ino‘, even though older docs mention them.

3. MQTT Topic Map

Use ‘<base>‘ as:

```
smartnest/SmartNest_001
```

Publish Topics: ESP32 -> Cloud

```
| Topic | Direction | Purpose | Trigger / frequency | Payload |
```

```
|---|---|---|---|---|
| ‘<base>/live/sensors‘ | SmartNest -> Cloud | Current sensor and energy readings | On MQTT connect, every 30 s hea
rtbeat, Master telemetry update, DHT update, selected command state changes | JSON object |
| ‘<base>/live/relays‘ | SmartNest -> Cloud | Relay states, locks, digital switch, runtimes | Same ‘publishLiveData
()‘ call as live sensors/status | JSON object |
| ‘<base>/live/status‘ | SmartNest -> Cloud | Device, network, MQTT, SD, slave, time, and data-quality status | Sam
e ‘publishLiveData()‘ call as live sensors/relays | JSON object |
| ‘<base>/history/batch‘ | SmartNest -> Cloud | SD-backed historical energy records | After MQTT connect and whenev
er unsent Master SD records are available; retries if ACK timeout | JSON object |
| ‘<base>/cmd/ack‘ | SmartNest -> Cloud | Command result acknowledgement | After command handling, Digital Board AC
K, invalid command payload, or timeout | JSON object |
```

Subscribe Topics: Cloud -> ESP32

```
| Topic | Direction | Purpose | Expected payload |
|---|---|---|---|
| ‘<base>/cmd/request‘ | Cloud -> SmartNest | Cloud command request | JSON command object |
| ‘<base>/history/ack‘ | Cloud -> SmartNest | Acknowledge a saved history batch | JSON ACK object |
```

4. Live Sensors Payload

Topic:

```
<base>/live/sensors
```

Actual fields published:

```
| Field | Type | Meaning |
|---|---|---|
| ‘voltage‘ | number | PZEM voltage from Master telemetry, formatted with 1 decimal. If PZEM data is unhealthy/offl
ine upstream, SmartNest state may hold ‘0.0‘. |
| ‘energy_voltage‘ | number | Voltage used by Master for energy calculations. When PZEM voltage is invalid, Master
falls back to ‘220.0‘ and sets ‘voltage_estimated=true‘. |
| ‘main_current‘ | number | SmartNest local relay 1-6 ACS current in amperes. |
| ‘digital_current‘ | number | Digital Board relay 7 ACS current in amperes. |
| ‘ac_current‘ | number | PZEM AC current in amperes. |
| ‘ac_power‘ | number | PZEM AC power in watts. |
| ‘ac_energy_kwh‘ | number | Daily AC energy calculated by Master as raw PZEM cumulative energy minus current day s
tart baseline. |
| ‘pzem_cumulative_energy_kwh‘ | number | Raw cumulative PZEM energy register value. |
| ‘ac_day_start_kwh‘ | number | Raw PZEM energy baseline captured for the current day. |
| ‘main_energy_kwh‘ | number | Integrated SmartNest relay 1-6 energy in kWh. |
| ‘digital_energy_kwh‘ | number | Integrated Digital Board relay 7 energy in kWh. |
| ‘temperature_c‘ | number | DHT11 temperature in Celsius. |
| ‘humidity_pct‘ | number | DHT11 relative humidity percentage. |
```

Important distinction:

- ‘voltage‘ is the direct PZEM voltage measurement exposed by telemetry.
- ‘energy_voltage‘ is the voltage used for energy math. It may be a fallback value.
- ‘voltage_estimated‘ is not published in ‘live/sensors‘; it is a status/data-quality flag and is published in ‘live/status‘.

Example payload:

```
{
  "voltage": 230.1,
  "energy_voltage": 230.1,
  "main_current": 1.25,
  "digital_current": 0.52,
  "ac_current": 1.234,
  "ac_power": 287.5,
  "ac_energy_kwh": 18.905,
  "pzem_cumulative_energy_kwh": 101.235,
  "ac_day_start_kwh": 82.330,
  "main_energy_kwh": 0.120,
  "digital_energy_kwh": 0.040,
  "temperature_c": 28.4,
  "humidity_pct": 61.0
}
```

Cloud storage guidance:

- Store live sensors if the cloud needs charts or recent device state.
- Treat live sensors as volatile snapshots; permanent billing/history should use ‘history/batch‘.
- Use ‘live/status.pzem_health‘ and ‘live/status.voltage_estimated‘ to qualify sensor trust.

5. Live Status Payload

Topic:

```
<base>/live/status
```

Actual fields published:

```
| Field | Type | Meaning |
|---|---|---|
| ‘uptime‘ | number | SmartNest ESP32 ‘millis()‘ value. |
| ‘ssid‘ | string | Connected Wi-Fi SSID. |
| ‘rssi‘ | number | SmartNest Wi-Fi RSSI. |
| ‘mqtt_status‘ | number | ‘0‘ disabled, ‘1‘ connecting, ‘2‘ connected, ‘3‘ error. |
| ‘sd_ok‘ | boolean | Master SD card status. |
| ‘sd_total‘ | number | Master SD total bytes. |
| ‘sd_used‘ | number | Master SD used bytes. |
| ‘digital_online‘ | boolean | Digital Board online status. |
| ‘pzem_online‘ | boolean | PZEM Board online status. |
| ‘pzem_health‘ | boolean | PZEM sensor read health. |
| ‘dht_ok‘ | boolean | SmartNest DHT11 health. |
| ‘voltage_estimated‘ | boolean | ‘true‘ when Master is using fallback voltage for energy calculations. |
| ‘time_source‘ | string | Master time source copied from UART ‘tsrc‘. Expected values from code: ‘NTP‘, ‘RTC‘, ‘SO
FT‘, ‘ESTIMATED‘, ‘NONE‘. |
| ‘reset_reason‘ | string | Reset reason copied from Master telemetry when available. |
```

Why these fields matter:

- ‘time_source‘: tells the cloud how reliable timestamps are. ‘NTP‘ is best, ‘RTC‘ is good offline, ‘SOFT‘
is derived from millis after valid time, and ‘ESTIMATED‘ is restored from SD state.
- ‘voltage_estimated‘: belongs in status because it is a quality flag, not a sensor reading.

- ‘pzem_health‘: when false, cloud should not blindly trust PZEM voltage/current/power.
- ‘sd_ok‘: if false, offline history buffering is at risk because Master cannot safely write SD history.

Example payload:

```
{
  "uptime": 123456,
  "ssid": "WiFiName",
  "rssi": -55,
  "mqtt_status": 2,
  "sd_ok": true,
  "sd_total": 3965190144,
  "sd_used": 1048576,
  "digital_online": true,
  "pzem_online": true,
  "pzem_health": true,
  "dht_ok": true,
  "voltage_estimated": false,
  "time_source": "NTP",
  "reset_reason": "POWERON"
}
```

6. Live Relays Payload

Topic:

```
<base>/live/relays
```

Actual fields published:

```
| Field | Type | Meaning |
|---|---|---|
| ‘states‘ | boolean array length 7 | Relay ON/OFF states. |
| ‘locks‘ | boolean array length 7 | Relay lock states. |
| ‘master_lock‘ | boolean | True only if Digital Board lock and all six local locks are true. |
| ‘digital_switch‘ | boolean | Digital Board manual switch state. |
| ‘runtime_sec‘ | number array length 7 | Relay ON runtime counters in seconds from Master telemetry. |
```

Relay index map:

```
| Array index | User relay | Device |
|---:|---:|---|
| ‘0‘ | Relay 1 | SmartNest / Internet ESP32 local relay |
| ‘1‘ | Relay 2 | SmartNest / Internet ESP32 local relay |
| ‘2‘ | Relay 3 | SmartNest / Internet ESP32 local relay |
| ‘3‘ | Relay 4 | SmartNest / Internet ESP32 local relay |
| ‘4‘ | Relay 5 | SmartNest / Internet ESP32 local relay |
| ‘5‘ | Relay 6 | SmartNest / Internet ESP32 local relay |
| ‘6‘ | Relay 7 | Digital Board relay |
```

Example payload:

```
{
  "states": [false, false, true, false, false, false, true],
  "locks": [false, false, false, false, false, false, false],
  "master_lock": false,
  "digital_switch": true,
  "runtime_sec": [12, 0, 44, 0, 0, 0, 18]
}
```

7. Command Request API over MQTT

Cloud sends all MQTT commands to:

```
<base>/cmd/request
```

The current code expects JSON. If JSON parsing fails, SmartNest publishes an ACK with ‘cmd_id="unknown"‘, ‘type="unknown"‘, ‘ok=false‘, and ‘reason="invalid_payload"‘.

Every command should include a unique ‘cmd_id‘. If missing, firmware generates a fallback ID like ‘cmd-<millis>‘, but cloud-side tracking is better if the cloud supplies one.

Supported command: ‘relay_set‘

Sets relay 1-7 ON/OFF.

```
{
  "cmd_id": "cmd-001",
  "type": "relay_set",
  "relay": 1,
  "state": true
}
```

Behavior:

- ‘relay‘ 1-6: SmartNest local relay is set directly.
- ‘relay‘ 7: SmartNest sends UART command to Master, which forwards ‘relay_on‘ or ‘relay_off‘ to Digital Board.
- If relay number is outside 1-7, ACK reason is ‘invalid_relay‘.
- If relay 7 already has a pending command, ACK reason is ‘busy‘.

Supported command: ‘relay_toggle‘

Toggles relay 1-7.

```
{
```

```
  "cmd_id": "cmd-002",
  "type": "relay_toggle",
  "relay": 7
}
```

Behavior:

- ‘relay‘ 1-6: toggled locally.
- ‘relay‘ 7: SmartNest reads the last known Digital Board relay state and sends the opposite command through Master.

Supported command: ‘relay_lock‘

Locks or unlocks relay 1-7.

```
{
  "cmd_id": "cmd-003",
  "type": "relay_lock",
  "relay": 3,
  "locked": true
}
```

Behavior:

- ‘relay‘ 1-6: SmartNest local lock is changed and saved.
- ‘relay‘ 7: command is forwarded to Digital Board as ‘relay_lock‘ or ‘relay_unlock‘.

Supported command: ‘master_lock‘

Applies the local lock-all alias.

```
{
  "cmd_id": "cmd-004",
  "type": "master_lock",
  "state": true
}
```

Behavior:

- SmartNest sets all six local relay locks to the requested state.
- SmartNest forwards ‘relay_lock‘ or ‘relay_unlock‘ to Digital Board.
- ACK reason is ‘alias_sent‘ when accepted.

Supported command: ‘slave_reboot‘

Reboots a slave board.

```
{
  "cmd_id": "cmd-005",
  "type": "slave_reboot",
  "target": "pzem"
}
```

Allowed targets:

- ‘digital‘
- ‘d1‘
- ‘pzem‘

Invalid targets return ‘invalid_target‘.

Supported command: ‘pzem_energy_reset‘

High-risk maintenance command.

```
{
  "cmd_id": "cmd-006",
  "type": "pzem_energy_reset"
}
```

Behavior:

- SmartNest sends ‘{"t":"cmd","tgt":"pzem","cmd":"energy_reset"}‘ over UART to Master.
- Master forwards ESP-NOW command type ‘0x07‘ to the PZEM slave.
- PZEM slave calls ‘pzem.resetEnergy()‘.

Risk:

- This can reset the raw cumulative PZEM energy register.
- Cloud UI should restrict this to an admin/maintenance workflow with confirmation.

Not implemented:

- Shutdown command over MQTT.
- MQTT command to change MQTT client ID.
- MQTT command to change MQTT base topic.
- Direct MQTT command for SD log deletion.
- Direct MQTT command for Wi-Fi reset.

8. Command ACK Procedure

SmartNest publishes command acknowledgements to:

```
<base>/cmd/ack
```

Actual ACK format:

```
{
  "cmd_id": "cmd-001",
  "type": "relay_set",
  "ok": true,
  "reason": "done",
  "relay": 1,
  "state": true,
  "locked": false
}
```

Fields:

```
| Field | Present | Meaning |
|---|---|---|
| ‘cmd_id‘ | always | Echoes command ID, or generated fallback. |
| ‘type‘ | always | Echoes command type or ‘unknown‘. |
| ‘ok‘ | always | Boolean command result. |
| ‘reason‘ | always | Machine-readable result reason. |
| ‘relay‘ | only when relay argument is supplied to ‘publishCommandAck()‘ | Relay number 1-7. |
| ‘state‘ | only with ‘relay‘ | Relay state after command, when known. |
| ‘locked‘ | only with ‘relay‘ | Lock state after command, when known. |
```

Actual reason values observed in code:

```
| Reason | Meaning |
|---|---|
| ‘done‘ | Command completed or final Digital Board ACK was successful. |
| ‘alias_sent‘ | ‘master_lock‘ alias was accepted and forwarded. |
| ‘sent‘ | Command was sent to another board; no final MQTT-level state payload is expected for that command. |
| ‘invalid_payload‘ | JSON parse failed. |
| ‘invalid_relay‘ | Relay number outside 1-7. |
| ‘invalid_target‘ | Slave reboot target was invalid. |
| ‘locked‘ | Local relay ON command did not turn on because relay was locked. |
| ‘overcurrent_locked‘ | Digital Board rejected command due to overcurrent lock. |
| ‘busy‘ | Relay 7 command already pending, or master-lock alias could not be sent. |
| ‘timeout‘ | Digital Board command ACK did not arrive within ‘15000‘ ms. |
| ‘rejected‘ | Command was not accepted for a reason other than mapped lock conditions. |
| ‘unsupported‘ | Command type is not implemented. |
```

Cloud matching strategy:

1. Generate a unique ‘cmd_id‘.
2. Publish command to ‘<base>/cmd/request‘.
3. Subscribe to ‘<base>/cmd/ack‘.
4. Match ACK by ‘cmd_id‘.
5. Mark command complete when ACK arrives.
6. If no ACK arrives within 15-20 seconds, show timeout in the cloud UI.

Retry guidance:

- Retry idempotent commands such as setting relay OFF only after the timeout window.

- Avoid blind retries for ‘relay_toggle‘, because repeating a toggle can undo the intended state.
- Avoid automatic retries for ‘pzem_energy_reset‘.

Example:

Cloud publishes:

```
{
  "cmd_id": "cmd-001",
  "type": "relay_set",
  "relay": 1,
  "state": true
}
```

SmartNest publishes:

```
{
  "cmd_id": "cmd-001",
  "type": "relay_set",
  "ok": true,
  "reason": "done",
  "relay": 1,
  "state": true,
  "locked": false
}
```

9. SD Card History Upload Procedure

History is designed to avoid data loss when cloud or MQTT is offline.

Implemented flow:

1. Master calculates energy and writes records to ‘/SmartNestLogs/energy_log.csv‘ on SD.
2. Internet ESP32 connects to MQTT.
3. Internet ESP32 requests history from Master over UART:

```
{"t":"hist_req","after":0,"limit":6}
```

4. Master replies over UART:

```
{"t":"hist_res","records":[...],"last":51}
```

5. Internet ESP32 publishes the records to:

```
<base>/history/batch
```

6. Cloud saves all records.
7. Cloud publishes ACK to:

```
<base>/history/ack
```

8. Internet ESP32 forwards ACK to Master:

```
{"t":"hist_ack","last":51}
```

9. Master writes ‘sync_state.txt‘ and purges acknowledged CSV rows with ‘record_id <= last‘.

Actual MQTT history batch payload:

```
{
  "batch_id": "SmartNest_001-51-123456",
  "device": "SmartNest_001",
  "records": [
    {
      "id": 51,
      "epoch": 1781971200,
      "date": "2026-06-20 22:10:00",
      "voltage": 230.1,
      "main_current": 1.250,
      "main_power_w": 287.63,
      "main_energy_kwh": 0.120400,
      "digital_current": 0.520,
      "digital_power_w": 119.65,
      "digital_energy_kwh": 0.040150,
      "ac_current": 1.234,
      "ac_power_w": 287.50,
      "ac_energy_kwh": 18.905000,
      "runtimes_sec": [12, 0, 44, 0, 0, 0, 18]
    }
  ]
}
```

Top-level fields:

```
| Field | Type | Meaning |
|---|---|---|
| ‘batch_id‘ | string | Built as ‘<clientId>-<lastRecordId>-<millis>‘. |
| ‘device‘ | string | Fixed MQTT client ID, currently ‘SmartNest_001‘. |
| ‘records‘ | array | History record objects from Master SD CSV. |
| ‘last_id‘ | Not implemented | The MQTT batch does not currently include top-level ‘last_id‘; derive the highest ‘
records[].id‘ or parse it from ‘batch_id‘ if needed. |
```

Record fields:

```
| Field | Type | Meaning |
```

```
|---|---|---|
| ‘id‘ | number | Monotonic SD ‘record_id‘. |
| ‘epoch‘ | number | UTC epoch seconds from Master time system. |
| ‘date‘ | string | Local date/time formatted by Master. |
| ‘voltage‘ | number/string in JSON source | Voltage stored in CSV. |
| ‘main_current‘ | number/string in JSON source | SmartNest local relay 1-6 current. |
| ‘main_power_w‘ | number/string in JSON source | ‘voltage * main_current‘. |
| ‘main_energy_kwh‘ | number/string in JSON source | Integrated local relay 1-6 energy. |
| ‘digital_current‘ | number/string in JSON source | Digital Board relay 7 current. |
| ‘digital_power_w‘ | number/string in JSON source | ‘voltage * digital_current‘. |
| ‘digital_energy_kwh‘ | number/string in JSON source | Integrated relay 7 energy. |
| ‘ac_current‘ | number/string in JSON source | PZEM AC current. |
| ‘ac_power_w‘ | number/string in JSON source | PZEM AC power. |
| ‘ac_energy_kwh‘ | number/string in JSON source | Daily AC energy from raw PZEM energy minus day baseline. |
| ‘runtimes_sec‘ | number array length 7 | Relay runtime counters for relay 1-7. |
| ‘time_source‘ | Not implemented in MQTT history record | Present in current CSV header, but omitted by ‘sendHisto
ryResponse()‘ when building MQTT history JSON. |
```

Recommended:

- Add ‘time_source‘ to each MQTT history record because it already exists in the SD CSV.
- Add top-level ‘last_id‘ to ‘history/batch‘ for easier ACK generation.
- Store history records idempotently by ‘(device, id)‘.

10. History ACK Procedure

Cloud acknowledges saved history batches on:

```
<base>/history/ack
```

Actual expected ACK payload:

```
{
  "batch_id": "SmartNest_001-51-123456",
  "ok": true,
  "last_id": 51
}
```

Required fields:

```
| Field | Required | Meaning |
|---|---|---|
| ‘batch_id‘ | yes | Must match SmartNest pending ‘batch_id‘. |
| ‘ok‘ | yes | Must be ‘true‘ for SmartNest to accept the ACK. |
| ‘last_id‘ | yes in cloud design; firmware falls back if ‘0‘ | Last saved record ID. If ‘0‘, SmartNest substitutes
 the pending batch last ID. |
```

Firmware validation:

- ACK is ignored unless ‘ok == true‘.
- ACK is ignored unless ‘batch_id‘ matches the currently pending batch.
- ACK is rejected if ‘last_id‘ is greater than the pending last record ID.
- On valid ACK, SmartNest sends ‘{"t":"hist_ack","last":<last_id>}‘ to Master.

- Master updates ‘sync_state.txt‘.
- Master purges acknowledged rows from ‘energy_log.csv‘.
- SmartNest immediately requests the next history batch.

Retry behavior:

- While a history batch is pending, SmartNest does not publish another batch.
- If no ACK arrives within ‘15000‘ ms, SmartNest clears the pending state and retries from the last acknowledged ID.
- Oversized batches are skipped and the requested batch limit is reduced.

Cloud ACK rules:

- Send ‘ok:true‘ only after every record in the batch has been saved successfully.
- If cloud storage fails, do not send ‘ok:true‘.
- Use idempotent upsert so duplicate retry batches do not create duplicate database rows.

Why ACK matters:

- Prevents deleting SD logs before cloud persistence.
- Enables retry after MQTT/cloud outage.
- Avoids data loss during offline operation.
- Lets Master safely purge acknowledged rows and keep SD storage bounded.

11. Cloud Database Design Recommendation

The following design separates volatile live state from permanent historical records and command audit data.

1. ‘devices‘

Suggested fields:

- ‘device_id‘ / ‘client_id‘
- ‘base_topic‘
- ‘broker‘
- ‘last_seen_at‘
- ‘mqtt_status‘
- ‘firmware_identity_readonly‘
- ‘created_at‘
- ‘updated_at‘

2. ‘live_sensor_readings‘

Suggested fields:

- ‘device_id‘
- ‘received_at‘

- ‘voltage‘
- ‘energy_voltage‘
- ‘main_current‘
- ‘digital_current‘
- ‘ac_current‘
- ‘ac_power‘
- ‘ac_energy_kwh‘
- ‘pzem_cumulative_energy_kwh‘
- ‘ac_day_start_kwh‘
- ‘main_energy_kwh‘
- ‘digital_energy_kwh‘
- ‘temperature_c‘
- ‘humidity_pct‘
- ‘quality_pzem_health‘
- ‘quality_voltage_estimated‘
- ‘source_topic‘

3. ‘live_status‘

Suggested fields:

- ‘device_id‘
- ‘received_at‘
- ‘uptime‘
- ‘ssid‘
- ‘rssi‘
- ‘mqtt_status‘
- ‘sd_ok‘
- ‘sd_total‘
- ‘sd_used‘
- ‘digital_online‘
- ‘pzem_online‘
- ‘pzem_health‘
- ‘dht_ok‘
- ‘time_source‘
- ‘voltage_estimated‘
- ‘reset_reason‘

4. ‘relay_states‘

Suggested fields:

- ‘device_id‘
- ‘received_at‘
- ‘relay_index‘
- ‘relay_number‘
- ‘state‘
- ‘locked‘
- ‘runtime_sec‘
- ‘digital_switch‘

- ‘master_lock‘
- ‘source_topic‘

5. ‘energy_history‘

Suggested fields:

- ‘device_id‘
- ‘record_id‘
- ‘batch_id‘
- ‘epoch‘
- ‘date‘
- ‘voltage‘
- ‘main_current‘
- ‘main_power_w‘
- ‘main_energy_kwh‘
- ‘digital_current‘
- ‘digital_power_w‘
- ‘digital_energy_kwh‘
- ‘ac_current‘
- ‘ac_power_w‘
- ‘ac_energy_kwh‘
- ‘r1_on_s‘
- ‘r2_on_s‘
- ‘r3_on_s‘
- ‘r4_on_s‘
- ‘r5_on_s‘
- ‘r6_on_s‘
- ‘r7_on_s‘
- ‘time_source‘ (Recommended, not currently present in MQTT history)
- ‘received_at‘

Use a unique key:

```
(device_id, record_id)
```

6. ‘command_logs‘

Suggested fields:

- ‘device_id‘
- ‘cmd_id‘
- ‘type‘
- ‘request_payload‘
- ‘ack_payload‘
- ‘requested_at‘
- ‘ack_at‘
- ‘ok‘

- ‘reason‘
- ‘relay‘
- ‘state‘
- ‘locked‘
- ‘timeout‘
- ‘requested_by‘

7. ‘history_batch_logs‘

Suggested fields:

- ‘device_id‘
- ‘batch_id‘
- ‘record_count‘
- ‘first_record_id‘
- ‘last_record_id‘
- ‘payload_bytes‘
- ‘received_at‘
- ‘saved_at‘
- ‘ack_sent_at‘
- ‘ack_ok‘
- ‘retry_count‘

Implementation Checklist For Cloud Developer

- Subscribe to ‘<base>/live/sensors‘, ‘<base>/live/status‘, ‘<base>/live/relays‘, and ‘<base>/history/batch‘.
- Publish commands only to ‘<base>/cmd/request‘.
- Publish history ACK only to ‘<base>/history/ack‘.
- Match command ACKs by ‘cmd_id‘.
- Match history ACKs by ‘batch_id‘.
- Store history idempotently by ‘(device, id)‘.
- Never ACK a history batch before records are durable in the database.
- Treat ‘pzem_energy_reset‘ as high-risk.
- Treat ‘time_source‘, ‘pzem_health‘, and ‘voltage_estimated‘ as data-quality signals.
- Do not use or create old compatibility topics such as ‘<base>/sensor/#‘, ‘<base>/relay/#‘, ‘<base>/status‘, or ‘<base>/cmd/slave/#‘.